

FASE 1 —PLAN TÉCNICO

1.1 Resumen ejecutivo

Qué construye NuevaMente: una aplicación web que recibe documentación técnica (PDF/Markdown/TXT), la indexa con RAG y la re-expresa como contenido educativo adaptado a un perfil, formato pedagógico, nicho y nivel de detalle — validando que lo generado esté anclado a la fuente antes de entregarlo.

Quién lo utilizará: un instructor, especialista técnico o diseñador instruccional (usuario único del MVP; no alumnos, no administradores, no LMS).

Qué problema resuelve: reescribir documentación técnica a mano para distintas audiencias es lento y no escala; hacerlo con un LLM sin control es propenso a alucinaciones.

Qué forma parte del MVP: ingesta (PDF/MD/TXT) → extracción → chunking → embeddings → vector store → retrieval → generación adaptada (perfil, formato, nicho, detalle) → validación de fidelidad con score → JSON estructurado → persistencia en OCI Object Storage → interfaz web para operar todo el flujo sin tocar la API directamente.

Qué queda fuera del MVP: autenticación/usuarios, app móvil, fine-tuning, múltiples proveedores LLM simultáneos, LMS, analítica avanzada, dashboard admin, video/voz, chatbot independiente, editor avanzado, sistema multiagente autónomo complejo, más de 3 perfiles / 3 formatos, multimodalidad, interpretación de diagramas, exportaciones (PDF/Anki/CSV), quiz interactivo en tiempo real, n8n en producción.

Qué significa “MVP terminado”: una persona externa al equipo sube un PDF por la interfaz web, elige perfil + formato + nivel de detalle, recibe contenido generado con score de fidelidad visible, y puede verificar que el original y el resultado quedaron guardados en OCI — todo sin usar la API directamente ni intervención técnica del equipo (ver Fase 18).

FASE 1.2 — Decisiones técnicas

Decisión	Clasificación
Proveedor de LLM/embeddings (Gemini propuesto, no confirmado)	☐ Bloqueante
Tamaño máximo de archivo de entrada	☐ Importante (se asume 10 MB, configurable)
Despliegue de n8n en VM de producción (OCI Compute)	☐ Puede resolverse después
Exportaciones (PDF/MD/Anki), quiz interactivo, multiagente	☐ No afecta al MVP
Documentos de referencia para los 3 escenarios de demo	☐ Importante (deben prepararse en Semana 5)

FASE 2 — REQUISITOS

CLASIFICACIÓN DE REQUISITOS

Código	Requisito	Clase
REQ-01	Ingesta de PDF	REQUISITO OBLIGATORIO
REQ-02	Ingesta de Markdown	REQUISITO OBLIGATORIO
REQ-03	Ingesta de texto plano	REQUISITO OBLIGATORIO
REQ-04	Extracción de contenido	REQUISITO OBLIGATORIO
REQ-05	Chunking	REQUISITO OBLIGATORIO
REQ-06	Embeddings	REQUISITO OBLIGATORIO
REQ-07	Vector Store	REQUISITO OBLIGATORIO
REQ-08	Retrieval / RAG	REQUISITO OBLIGATORIO
REQ-09	LLM de generación	REQUISITO OBLIGATORIO
REQ-10	Adaptación por perfil	REQUISITO OBLIGATORIO
REQ-11	Adaptación por formato pedagógico	REQUISITO OBLIGATORIO
REQ-12	Adaptación por nicho/contexto	REQUISITO OBLIGATORIO
REQ-13	Nivel de detalle configurable	REQUISITO OBLIGATORIO
REQ-14	Validación de fidelidad	REQUISITO OBLIGATORIO
REQ-15	Metadatos educativos	REQUISITO OBLIGATORIO
REQ-16	JSON estructurado (schema formal)	REQUISITO OBLIGATORIO
REQ-17	Interfaz o API funcional (Streamlit/Gradio/FastAPI/Flask)	REQUISITO OBLIGATORIO
REQ-18	OCI Object Storage (Always Free)	REQUISITO OBLIGATORIO
REQ-19	Persistencia de documentos originales	REQUISITO OBLIGATORIO
REQ-20	Persistencia de contenido generado	REQUISITO OBLIGATORIO
REQ-21	Tres escenarios de demostración	REQUISITO OBLIGATORIO
REQ-22	README	REQUISITO OBLIGATORIO
REQ-23	Arquitectura documentada	REQUISITO OBLIGATORIO
REQ-24	Al menos 2 formatos pedagógicos distintos en el MVP	REQUISITO OBLIGATORIO
DIF-01	OCI Compute	DIFERENCIAL / OPCIONAL
DIF-02	Multi-Agent con LangGraph	DIFERENCIAL / OPCIONAL

Código	Requisito	Clase
DIF-03	Agente Investigador RAG	DIFERENCIAL / OPCIONAL
DIF-04	Agente Redactor Pedagógico	DIFERENCIAL / OPCIONAL
DIF-05	Agente Crítico/Revisor	DIFERENCIAL / OPCIONAL
DIF-06	Quiz interactivo con evaluación en tiempo real	DIFERENCIAL / OPCIONAL
DIF-07	Soporte multimodal	DIFERENCIAL / OPCIONAL
DIF-08	Interpretación de diagramas	DIFERENCIAL / OPCIONAL
DIF-09	Exportación PDF	DIFERENCIAL / OPCIONAL
DIF-10	Exportación Markdown	DIFERENCIAL / OPCIONAL
DIF-11	Exportación CSV/Anki	DIFERENCIAL / OPCIONAL

2.1 Requisitos funcionales

Formato completo para los requisitos que definen el Vertical Slice (críticos); el resto en tabla compacta por espacio — todos son igualmente exigibles.

RF-001
Nombre: Ingesta de documento técnico
Descripción: El sistema permite subir un archivo PDF, Markdown o TXT como fuente de conocimiento.
Actor: Usuario (instructor/especialista técnico)
Precondiciones: Ninguna
Flujo principal:
1. Usuario selecciona un archivo desde la interfaz web
2. Sistema valida tipo y tamaño (≤10 MB, DT-12)
3. Sistema sube el original a OCI Object Storage
4. Sistema genera un document_id y lo retorna
Flujos alternativos:
- Archivo inválido/corrupto → error controlado, no se sube
- Archivo excede tamaño máximo → error controlado
- PDF sin texto (escaneado) → advertencia explícita, no crash
Resultado esperado: documento almacenado en OCI con document_id trazable
Criterios de aceptación:
- PDF, MD y TXT se ingieren correctamente
- document_id es único y recuperable
- Todos los casos de error tienen mensaje explícito, no excepción no controlada
Prioridad: P0 (MUST_HAVE)
Dependencias: –

RF-009
Nombre: Generación de contenido adaptado

Descripción: El sistema genera contenido educativo a partir del contexto recuperado por RAG, adaptado a perfil, formato, nicho y nivel de detalle.

Actor: Usuario

Precondiciones: Documento ya ingerido e indexado (RF-001..RF-08)

Flujo principal:

1. Usuario elige perfil, formato, nicho (opcional) y nivel de detalle
2. Sistema recupera contexto relevante (RF-08)
3. Sistema construye el prompt y llama al LLM
4. Sistema retorna contenido_adaptado

Flujos alternativos:

- Sin contexto relevante (score de retrieval bajo) → el sistema no genera libremente, retorna status NO_CONTEXT

Resultado esperado: contenido educativo coherente con el perfil/formato elegido, anclado al documento fuente

Criterios de aceptación:

- El mismo documento con distinto perfil produce salidas perceptiblemente distintas
- El mismo documento con distinto formato produce estructuras distintas
- Nunca se genera contenido sin contexto mínimo verificado

Prioridad: P0 (MUST_HAVE)

Dependencias: RF-05, RF-06, RF-07, RF-08

RF-014

Nombre: Validación de fidelidad

Descripción: El sistema identifica afirmaciones del contenido generado y calcula un score de cuánto está sustentado en el documento fuente.

Actor: Sistema (automático, post-generación)

Precondiciones: Contenido generado (RF-09)

Flujo principal:

1. Sistema extrae claims del contenido generado
2. Sistema contrasta cada claim contra los chunks fuente usados
3. Sistema calcula fidelidad_score en [0,1]
4. Sistema lista claims no sustentados, si existen

Flujos alternativos:

- Fallo del LLM-juez → score=null + advertencia explícita, no bloquea la entrega del resultado

Resultado esperado: score reproducible + lista de observaciones incluida en el JSON final

Criterios de aceptación:

- Score siempre en rango [0,1] cuando se calcula
- Claims no sustentados quedan listados explícitamente
- El sistema nunca afirma "0% de alucinaciones"

Prioridad: P0 (MUST_HAVE)

Dependencias: RF-09

Requisitos funcionales (formato compacto):

ID	Nombre	Prioridad	Dependencias
RF-002	Ingesta de Markdown	P0	—
RF-003	Ingesta de TXT	P0	—
RF-004	Extracción de texto crudo	P0	RF-001/002/003
RF-005	Chunking con metadata	P0	RF-004
RF-006	Generación de embeddings	P0	RF-005
RF-007	Persistencia en Vector Store	P0	RF-006
RF-008	Retrieval semántico (top-k)	P0	RF-007
RF-010	Adaptación por perfil	P0	RF-009
RF-011	Adaptación por nicho/contexto (parámetro libre)	P0	RF-009
RF-012	Adaptación por formato pedagógico	P0	RF-009
RF-013	Selección de nivel de detalle (breve/estándar/profundo)	P0	RF-009
RF-015	Metadata educativa (conceptos clave, prerrequisitos, dificultad, tiempo estimado)	P0	RF-009
RF-016	Salida en JSON estructurado (schema Pydantic)	P0	RF-009, RF-014, RF-015
RF-017	Persistencia de original y resultado en OCI Object Storage	P0	RF-001, RF-016
RF-018	Visualización del resultado en interfaz web	P0	RF-016

2.2 Requisitos no funcionales

RNF-001 – Seguridad

Ninguna API key, secret o credencial OCI se almacena en el código fuente. Se usa `.env` + `.env.example`.

RNF-002 – Disponibilidad (MVP)

No se requiere alta disponibilidad; el MVP corre en un solo proceso local o en una VM. Objetivo: disponible durante la ventana de demo.

RNF-003 – Observabilidad

Cada ejecución del pipeline genera logs estructurados con: `document_id`, `etapa`, `duración`, `resultado` (éxito/error). Suficiente para depurar sin herramientas externas.

RNF-004 – Privacidad

Los documentos y contenidos generados se almacenan únicamente en el bucket OCI del equipo (Always Free); no se comparten con terceros fuera del flujo definido.

RNF-005 – Mantenibilidad

Cada capa (ingestion, processing, embeddings, rag, generation, validation, output, storage) es un módulo independiente con interfaz propia, para poder sustituir el proveedor de LLM/embeddings sin tocar el resto del pipeline (ver DT-09).

RNF-006 – Trazabilidad

Todo contenido generado debe poder relacionarse con document_id, chunk_id, posición y (cuando aplique) página/sección de origen.

RNF-007 – Costos

Todos los servicios usados deben estar dentro de capa gratuita (OCI Always Free, Gemini free tier, librerías open source). Ningún servicio de pago se activa sin aprobación explícita del equipo.

RNF-008 – Rendimiento (MVP)

Una generación completa (ingesta ya indexada → resultado visible) debe completarse en un tiempo razonable para demo en vivo (objetivo orientativo: <60s por generación; no es un requisito duro, es una guía de diseño).

FASE 3 — CASOS DE USO

CU-001

Nombre: Generar contenido educativo adaptado desde un documento nuevo

Actor: Usuario (instructor/especialista técnico)

Objetivo: Obtener contenido educativo personalizado a partir de un documento técnico propio

Precondiciones: Ninguna (primer uso del documento)

Entrada: archivo (PDF/MD/TXT), perfil, formato, nicho (opcional), nivel de detalle

Proceso: Ingesta → Extracción → Chunking → Embeddings → Vector Store → Retrieval → Generación → Validación → JSON → Persistencia OCI

Salida: contenido_adaptado + score de fidelidad + referencias a fuente, visible en la interfaz web

Errores posibles: archivo inválido, archivo vacío, documento sin texto extraíble, fallo del proveedor LLM/embeddings, fallo de red hacia OCI

Dependencias: RF-001 a RF-018

CU-002

Nombre: Generar el mismo documento para una audiencia distinta

Actor: Usuario

Objetivo: Reutilizar un documento ya indexado para producir contenido con otro perfil/formato

Precondiciones: El documento ya fue ingerido e indexado (CU-001 ejecutado antes)

Entrada: document_id existente, nuevo perfil/formato/nicho/detalle

Proceso: Retrieval (reutiliza embeddings existentes) → Generación → Validación → JSON → Persistencia

Salida: nuevo contenido_adaptado distinto al anterior para el mismo documento

Errores posibles: document_id inexistente
Dependencias: RF-008 a RF-018

CU-003
Nombre: Recibir aviso de fidelidad baja
Actor: Usuario
Objetivo: Ser advertido cuando el contenido generado no está bien sustentado en la fuente
Precondiciones: Contenido ya generado
Entrada: contenido_adaptado + chunks fuente
Proceso: Validación de fidelidad (RF-014)
Salida: score bajo el umbral configurado + lista de claims no sustentados, visible antes de que el usuario use el contenido
Errores posibles: fallo del LLM-juez (se informa, no se oculta)
Dependencias: RF-014

Trazabilidad Requisito → Caso de uso → Componente → Tarea (extracto — la matriz completa está en Fase 9/sección 14):

Requisito	Caso de uso	Componente	Tarea
RF-001	CU-001	COMP-01 Ingestion	BE-ING-001, BE-ING-002
RF-009	CU-001, CU-002	COMP-06 Generation	BE-GEN-003
RF-014	CU-001, CU-003	COMP-07 Validation	BE-VAL-002
RF-017	CU-001	COMP-09 Storage	OCI-003, OCI-005

FASE 4 — ARQUITECTURA

4.1 Arquitectura lógica

flowchart TD
U[Usuario] --> UI[Interfaz web - Streamlit]
UI -->|HTTP REST| API[Monolito backend - FastAPI]
API --> ING[Ingestion]
ING --> PROC[Processing / Chunking]
PROC --> EMB[Embeddings]
EMB --> VS[Vector Store - ChromaDB embebido]
VS --> RAG[RAG / Retrieval]
RAG --> GEN[Generation - LLM]
GEN --> VAL[Validation - Fidelidad]
VAL --> OUT[Output - JSON Pydantic]
OUT --> ST[Storage - OCI Object Storage]
ST --> UI

4.2 Arquitectura física

```
flowchart LR
    subgraph Local["Máquina local / VM (proceso único del monolito)"]
        UIProc["proceso: streamlit run app.py"]
        APIProc["proceso: uvicorn src.api.main:app"]
        Chroma["ChromaDB embebido (mismo proceso que APIProc)"]
        SQLite["SQLite local (estado de sesión)"]
    end
    UIProc -->|HTTP localhost| APIProc
    APIProc --> Chroma
    APIProc --> SQLite
    APIProc -->|HTTPS| Gemini["Gemini API (externo)"]
    APIProc -->|HTTPS| OCI["OCI Object Storage (externo, Always Free)"]
```

Nota: son dos procesos (UI y API), no uno solo — pero el backend (API) es un **monolito** porque toda la lógica de negocio (ingestion, RAG, generación, validación, output) vive en un único servicio, sin descomponerse en microservicios independientes. ChromaDB corre embebido dentro del proceso del API (no es un servidor aparte), lo que refuerza el carácter monolítico del backend.

4.3 Flujo de datos

```
sequenceDiagram
    participant U as Usuario
    participant UI as Interfaz web
    participant API as API (monolito)
    participant OCI as OCI Object Storage
    participant VS as Vector Store
    participant LLM as LLM (Gemini)

    U->>UI: sube archivo + elige perfil/formato
    UI->>API: POST /ingest
    API->>OCI: guarda original
    API-->>UI: document_id
    UI->>API: POST /adapt (document_id, perfil, formato)
    API->>VS: retrieval top-k
    API->>LLM: prompt + contexto
    LLM-->>API: contenido generado
    API->>LLM: verificación de claims (fidelidad)
    API->>OCI: guarda JSON resultado
    API-->>UI: JSON final (contenido + score)
    UI-->>U: muestra resultado
```

4.4 Flujo de usuario

Subir documento → elegir perfil/formato/nicho/detalle → esperar generación (loading) → ver resultado con score de fidelidad → (opcional) repetir con otro perfil sobre el mismo documento (CU-002).

4.5 Flujo de IA

Uso de IA (generativa)

Generar el contenido adaptado (RF-009) — requiere creatividad controlada

Extraer claims del contenido generado (RF-014) — requiere comprensión de lenguaje

Juzgar si un claim está sustentado en el contexto (RF-014) — requiere razonamiento

FASE 5 — COMPONENTES

COMP-01

Nombre: Ingestion Service

Responsabilidad: Recibir archivo, detectar tipo, extraer texto crudo, normalizar, validar tamaño/formato

Tecnología: Python, PyPDF, parsers MD/TXT propios

Entrada: bytes del archivo + nombre

Salida: {raw_text, file_type, source_metadata}

Dependencias: COMP-09 (Storage, para guardar el original)

API: función interna `ingest(file) -> IngestResult`, expuesta como parte de POST /ingest

Archivos: backend/src/ingestion/*

Responsable: rol Backend Ingesta

Riesgos: PDFs escaneados sin texto (mitigado con advertencia explícita, RF-001 flujo alternativo)

Pruebas: QA-001

COMP-02

Nombre: Processing Service

Responsabilidad: Limpieza de texto y chunking con metadata de posición/sección

Tecnología: LangChain RecursiveCharacterTextSplitter

Entrada: raw_text normalizado

Salida: list[DocumentChunk]

Dependencias: COMP-01

API: función interna, no expuesta como endpoint propio

Archivos: backend/src/processing/*

Responsable: rol Backend Processing

Riesgos: documentos muy grandes (mitigado con PROC-005)

Pruebas: QA-002 (parcial)

COMP-03

Nombre: Embedding Service

Responsabilidad: Generar vectores para cada chunk vía proveedor configurable

Tecnología: Interfaz EmbeddingProvider + implementación Gemini (DT-09, PENDIENTE DE VALIDACIÓN)

Entrada: list[str] (texto de los chunks)

Salida: list[vector]

Dependencias: COMP-02

API: interna, interfaz `EmbeddingProvider.embed(texts)`

Archivos: backend/src/embeddings/*

Responsable: rol Backend RAG

Riesgos: dependencia de un proveedor externo aún no confirmado – mitigado con mock intercambiable por flag

Pruebas: QA-002

COMP-04

Nombre: Vector Store

Responsabilidad: Persistir embeddings + metadata, realizar búsqueda semántica

Tecnología: ChromaDB embebido (persistente en disco, sin servidor externo)

Entrada: (chunk_id, vector, metadata, texto)

Salida: top-k chunks relevantes dado un vector de consulta

Dependencias: COMP-03

API: interna

Archivos: backend/src/vectorstore/*

Responsable: rol Backend RAG

Riesgos: colección vacía / no inicializada (mitigado con OUT-005)

Pruebas: QA-002

COMP-05

Nombre: RAG / Retrieval Service

Responsabilidad: Construir query desde perfil/nicho, recuperar contexto top-k, ensamblarlo para el LLM

Tecnología: Python

Entrada: {document_id, perfil, nicho, tema opcional}

Salida: contexto ensamblado con referencias a chunk_id

Dependencias: COMP-03, COMP-04

API: interna

Archivos: backend/src/rag/*

Responsable: rol Backend RAG

Riesgos: ausencia de contexto relevante (mitigado con BE-RAG-011, retorna NO_CONTEXT en vez de alucinar)

Pruebas: QA-003 (parcial, vía integración)

COMP-06

Nombre: Generation Service

Responsabilidad: Adaptar contenido por perfil/formato/nicho/nivel de detalle vía LLM

Tecnología: Interfaz LLMPProvider + implementación Gemini (DT-09, PENDIENTE DE VALIDACIÓN)

Entrada: {contexto, perfil, formato, nicho, nivel_detalle}

Salida: contenido_adaptado (texto/estructura según formato)

Dependencias: COMP-05

API: interna, parte de POST /adapt

Archivos: backend/src/generation/*

Responsable: rol Backend Generación

Riesgos: coste/latencia del LLM en demo en vivo (mitigado con RNF-008 como guía, no requisito duro)

Pruebas: QA-003

COMP-07

Nombre: Validation Service

Responsabilidad: Extraer claims, contrastarlos contra el contexto fuente, calcular score de fidelidad

Tecnología: LLM como "juez" + agregación determinista del score (DT-05)

Entrada: contenido_adaptado + chunks fuente usados

Salida: {fidelidad_score, claims_no_soportados, observaciones}

Dependencias: COMP-06

API: interna

Archivos: backend/src/validation/*

Responsable: rol Backend Validación

Riesgos: fallo del LLM-juez (mitigado con VAL-005, degrada a score=null sin bloquear la entrega)

Pruebas: QA-004 (golden test)

COMP-08

Nombre: Output Service

Responsabilidad: Ensamblar el JSON final según schema Pydantic formal

Tecnología: Pydantic v2 (DT-06)

Entrada: contenido_adaptado + evaluación de fidelidad + metadatos educativos

Salida: instancia validada de NuevaMenteOutput (JSON)

Dependencias: COMP-06, COMP-07

API: interna

Archivos: backend/src/output/*

Responsable: rol Backend Output

Riesgos: campos opcionales mal definidos (mitigado con BE-OUT-002 y sus tests)

Pruebas: incluidas en QA-003

COMP-09

Nombre: Storage Service

Responsabilidad: Cliente OCI, upload/download de originales y resultados, naming convention, verificación

Tecnología: OCI SDK (Object Storage, Always Free)
Entrada: bytes (original o JSON) + document_id
Salida: {bucket, object_name, url, uploaded_at}
Dependencias: –
API: interna, usada por COMP-01 (subida de original) y COMP-08 (subida de resultado)
Archivos: backend/src/storage/*
Responsable: rol Cloud/OCI
Riesgos: fallo de red hacia OCI (mitigado con verificación post-upload, status PARTIAL si falla)
Pruebas: incluidas en QA-003

COMP-10
Nombre: Interfaz web (Streamlit)
Responsabilidad: Permitir a un usuario no técnico ejecutar todo el flujo sin usar la API directamente
Tecnología: Streamlit
Entrada: interacción de usuario
Salida: visualización de resultado + score de fidelidad + estado de OCI
Dependencias: COMP-11 (API pública del monolito)
API: consume la API REST del monolito (no expone API propia)
Archivos: ui/*
Responsable: rol UX/UI + Frontend
Riesgos: usuario no técnico se confunde con errores crudos de la API (mitigado con mensajes amigables, UX-001)
Pruebas: prueba manual guiada (Fase 16, "Frontend tests")

COMP-11
Nombre: API pública (FastAPI)
Responsabilidad: Exponer el monolito vía REST; orquesta las llamadas a COMP-01..COMP-09 en orden
Tecnología: FastAPI + Pydantic
Entrada: requests HTTP (multipart para archivos, JSON para parámetros)
Salida: responses JSON según contratos de Fase 6
Dependencias: todos los componentes anteriores
API: ver Fase 6
Archivos: backend/src/api/*
Responsable: rol Orquestación/API
Riesgos: acoplar demasiada lógica de negocio en el controlador (mitigado: main.py solo orquesta, la lógica vive en cada Service)
Pruebas: QA-003

FASE 6 — CONTRATOS ENTRE COMPONENTES

POST /ingest
Request (multipart/form-data):

file: binary

Response 201:

```
{
  "document_id": "doc_a1b2c3",
  "file_type": "pdf",
  "status": "INGESTED"
}
```

Error 400 (archivo inválido/corrupto/tamaño excedido):

```
{
  "error": {
    "code": "INVALID_FILE",
    "message": "El archivo excede el tamaño máximo permitido (10 MB)"
  }
}
```

POST /adapt

Request (application/json):

```
{
  "document_id": "doc_a1b2c3",
  "perfil": "principiante",
  "formato": "flashcards",
  "nicho": "cloud computing",
  "nivel_detalle": "estandar"
}
```

Response 202 (procesamiento asíncrono):

```
{
  "job_id": "job_x9y8z7",
  "status": "PROCESSING"
}
```

Error 404 (document_id inexistente):

```
{
  "error": {
    "code": "DOCUMENT_NOT_FOUND",
    "message": "No existe un documento indexado con ese document_id"
  }
}
```

GET /adapt/{job_id}

Response 200 (según schema Pydantic completo, ver Fase 5 COMP-08):

```
{
  "status": "SUCCESS",
  "metadatos": { "doc_id": "...", "perfil": "...", "formato": "...", "chunks_fuente": ["chunk_01", "chunk_02"] },
}
```

```
"contenido_adaptado": { "cuerpo": "...", "conceptos_clave": [...], "prerrequisitos": [...], "tiempo_estimado_min": 15 },
"evaluacion_calidad": { "fidelidad_score": 0.93, "claims_no_soportados": [], "dificultad": "baja" },
"almacenamiento_oci": { "bucket": "nuevamente-g10", "object_name_json": "generated/doc_a1b2c3/...", "uploaded_at": "..." }
}
Response 200 (status alternativo, sin contexto suficiente):
{ "status": "NO_CONTEXT", "metadatos": { "...": "..." } }
Error 500 (fallo interno no recuperable):
{ "error": { "code": "INTERNAL_ERROR", "message": "..." } }

GET /health
Response 200:
{ "status": "ok" }
```

Variables de configuración (contrato de entorno, ver .env.example):

```
LLM_PROVIDER=gemini           # o "mock"
EMBEDDING_PROVIDER=gemini      # o "mock"
GEMINI_API_KEY=                # PENDIENTE DE VALIDACIÓN – proveedor no confirmado
FIDELITY_THRESHOLD=0.9         # DT-11
MAX_FILE_SIZE_MB=10           # DT-12, supuesto
OCI_CONFIG_FILE=~/.oci/config
OCI_BUCKET_NAME=nuevamente-g10
CHROMA_PERSIST_DIR=./data/chroma
```

Con este contrato publicado (Sprint 1), la Interfaz web puede construirse contra /adapt mockeado mientras Backend termina la lógica real — igual que en el plan original.

FASE 7 — ESTRUCTURA DEL PROYECTO

Adaptada a las tecnologías reales (Python monolito + Streamlit, sin frontend SPA):

```
nuevamente-g10/
├── .env.example
├── .gitignore
├── README.md
├── demo.sh
├── backend/                # monolito – toda la lógica de negocio
│   ├── pyproject.toml
│   ├── src/
│   │   └── core/           # config, logging
```

```

├── ingestion/      # COMP-01
├── processing/     # COMP-02
├── embeddings/     # COMP-03
├── vectorstore/    # COMP-04
├── rag/            # COMP-05
├── generation/     # COMP-06
├── validation/     # COMP-07
├── output/         # COMP-08
├── storage/        # COMP-09
├── db/             # estado de sesión (SQLite)
├── api/            # COMP-11 – orquesta, no contiene lógica de negocio
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── ui/             # COMP-10 – interfaz web, NO es un frontend SPA
│   ├── app.py
│   ├── components/
│   └── assets/
├── n8n-workflows/  # diferencial, ver DT-10
├── docs/
│   ├── architecture.md
│   ├── diagrams/
│   └── demo-scenarios/
├── data/           # gitignored – estado local (chroma, sqlite)
```

*(El árbol completo con cada archivo mapeado a su tarea

Estructura derivada de las capas de arquitectura (Ingestion → Processing → Embeddings → Vector Store → RAG → Generation → Validation → Output → Storage → Interface) y de las decisiones técnicas DT-01...DT-11.

```

nuevamente-g10/
├── .env.example    # DT-09, DT-11, DT-12 – variables sin secretos
├── .gitignore      # venv/, .env, data/chroma, data/mock_oci
├── README.md       # DOC-001
├── demo.sh         # guion de demo ejecutable (DEMO-001..003)
├── docker-compose.yml # (opcional) levanta backend + ui + n8n local
├── backend/
│   └── pyproject.toml # FND-002
```

```
— requirements.txt
— src/
  — core/                                # FND-001, FND-003, FND-004
    — config.py                          # pydantic-settings, carga .env
    — logging.py
  — ingestion/                           # EPIC-02
    — router.py                          # BE-ING-001
    — validators.py                      # ING-005, ING-006
    — pdf_extractor.py                   # BE-ING-002, ING-007
    — md_extractor.py                    # BE-ING-003, ING-008
    — txt_extractor.py                   # BE-ING-004, ING-009
  — processing/                           # EPIC-03
    — models.py                          # BE-PROC-001 (DocumentChunk)
    — cleaning.py                        # PROC-003
    — chunking.py                       # BE-PROC-002, PROC-004, PROC-005
  — embeddings/                           # EPIC-04
    — base.py                           # interfaz EmbeddingProvider
    — provider_gemini.py                 # BE-RAG-004, DT-09
    — provider_mock.py                   # mock (sección Mocks)
    — embed_chunks.py                   # BE-RAG-005, OUT-004
  — vectorstore/                          # EPIC-04
    — client.py                          # BE-RAG-006, DT-02
    — store.py                           # BE-RAG-007
    — search.py                          # BE-RAG-008, OUT-005
  — rag/                                  # EPIC-05
    — retrieval_service.py               # BE-RAG-009
    — context_builder.py                 # BE-RAG-010, BE-RAG-011
  — generation/                           # EPIC-06
    — orchestrator.py                   # BE-GEN-003, GEN-004, GEN-005
    — llm_provider.py                   # LLMProvider (Gemini/mock), DT-09
    — prompts/
      — perfiles/                        # GEN-001, GEN-007
      — formatos/                        # GEN-002, GEN-006
  — validation/                           # EPIC-07
```

claims_extractor.py	# VAL-001
fidelity_checker.py	# BE-VAL-002, DT-05, VAL-005
pedagogical_eval.py	# VAL-003, VAL-004
output/	# EPIC-08
schema.py	# BE-OUT-002, DT-06
assembler.py	# BE-OUT-003
storage/	# EPIC-09
oci_config.py	# OCI-001
oci_client.py	# OCI-002, DT-07
upload.py	# OCI-003, OCI-005
verify.py	# OCI-005
db/	# DT-08 – estado de sesión
models.py	
session.py	
api/	# EPIC-10, EPIC-12
main.py	# FE-API-001 (mock) → FE-API-002 (real)
schemas.py	
orchestrator.py	# INT-001, INT-002, INT-003
tests/	# EPIC-11
unit/	# QA-001, QA-002
integration/	# QA-003, QA-004
fixtures/	# documentos de prueba (golden tests)
ui/	# EPIC-10 – Streamlit
app.py	# FE-UI-005
components/	
upload_view.py	# UX-001 (pantalla 1)
selection_view.py	# UX-001 (pantalla 2)
loading_view.py	# UX-001 (pantalla 3)
result_view.py	# UX-001 (pantalla 4)
error_view.py	# UX-001 (pantalla 5)
assets/	
n8n-workflows/	# DT-10 (COULD – Semana 5)
nuevamente-v1.json	# INT-004

```
docs/                                # EPIC-13
├── architecture.md                  # DOC-002
├── diagrams/
│   └── pipeline.mmd                # diagrama Mermaid de la arquitectura
└── demo-scenarios/                 # DEMO-001, DEMO-002, DEMO-003
    ├── escenario-1-principiante.md
    ├── escenario-2-lider-tecnico.md
    └── escenario-3-formato-alterno.md

data/                                # gitignored – estado local de desarrollo
├── chroma/                         # persistencia ChromaDB (DT-02)
├── mock_oci/                       # mock de OCI upload (sección Mocks)
└── sqlite/                         # DT-08
```

FASE 8 — BACKLOG (EPIC → Feature → Task → Subtask)

BACKLOG DE ARRANQUE — 4 semanas de desarrollo + 1 semana de pruebas/ajustes

SEMANA 1 — Foundation + Ingesta + contrato API (mock)

Corresponde a FASE 1 + FASE 2. Objetivo de cierre: repo funcional, ingesta probada, /adaptar mockeado, mockups de UI listos.

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
FND-001	Crear estructura de carpetas src/, tests/, docs/	Foundation	MUST	XS	—	✓
FND-002	Configurar pyproject.toml/requirements.txt	Foundation	MUST	XS	FND-001	✓
FND-003	Configurar .env.example y loader de settings	Foundation	MUST	XS	FND-001	✓
FND-004	Configurar logging estructurado	Foundation	SHOULD	XS	FND-001	—
FND-005	Configurar linter/formatter + pre-commit	Foundation	SHOULD	XS	FND-002	—
SEC-001	Auditar repo: sin API keys/secrets hardcodedos	Foundation	MUST	XS	FND-003	—

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
BE-ING-001	Detectar tipo de archivo y enrutar a extractor	Backend Ingesta	MUST	XS	FND-001	✓
BE-ING-002	Extraer texto de PDF	Backend Ingesta	MUST	S	BE-ING-001	✓
BE-ING-003	Extraer/normalizar texto de Markdown	Backend Ingesta	MUST	XS	BE-ING-001	✓
BE-ING-004	Extraer/normalizar texto plano	Backend Ingesta	MUST	XS	BE-ING-001	✓
ING-005	Validar tamaño máximo de archivo (DT-12: 10 MB)	Backend Ingesta	MUST	XS	BE-ING-001	—
ING-006	Manejo de error: archivo inválido/corrupto	Backend Ingesta	MUST	XS	BE-ING-002	—
ING-007	Manejo de error: PDF sin texto (escaneado)	Backend Ingesta	MUST	S	BE-ING-002	—
ING-008	Manejo de error: Markdown vacío	Backend Ingesta	MUST	XS	BE-ING-003	—
ING-009	Manejo de error: texto vacío	Backend Ingesta	MUST	XS	BE-ING-004	—
BE-PROC-001	Definir estructura DocumentChunk	Backend Processing	MUST	XS	FND-001	✓
PROC-003	Limpieza de texto	Backend Processing	MUST	S	BE-ING-002..004	✓
BE-PROC-002	Implementar chunking	Backend Processing	MUST	S	BE-PROC-001, PROC-003	✓
PROC-004	Metadata de sección/posición en chunk	Backend Processing	MUST	XS	BE-PROC-002	—
PROC-005	Manejo de error: documento demasiado grande	Backend Processing	MUST	S	PROC-003	—
BE-OUT-002	Definir schema Pydantic de salida	Backend Output	MUST	S	FND-001	—

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
FE-API-001	Contrato REST mockeado (/ingest, /adapt)	Orquestación/API	MUST	XS	FND-001	✓
UX-001	Mockups de las 5 pantallas de Streamlit	UX/UI	MUST	S	FND-001	—
OCI-001	Configurar credenciales OCI de forma segura	Cloud/OCI	MUST	S	FND-001	—

Todas las tasks siguen la misma plantilla completa de la sección 15 del brief; aquí se listan en tabla por espacio. CRIT = pertenece al critical path (sección 7).

ID	Título	Epic	Prioridad	Est.	Dependencias	Crit
FND-001	Crear estructura de carpetas src/, tests/, docs/	EPIC-01	MUST	XS	—	✓
FND-002	Configurar pyproject.toml/requirements.txt	EPIC-01	MUST	XS	FND-001	✓
FND-003	Configurar .env.example y loader de settings (pydantic-settings)	EPIC-01	MUST	XS	FND-001	✓
FND-004	Configurar logging estructurado (structlog/logging)	EPIC-01	SHOULD	XS	FND-001	—
FND-005	Configurar linter/formatter (ruff/black) + pre-commit	EPIC-01	SHOULD	XS	FND-002	—
ING-005	Validar tamaño máximo de archivo	EPIC-02	MUST	XS	BE-ING-001	—
ING-006	Manejo de error: archivo inválido/corrupto	EPIC-02	MUST	XS	BE-ING-002	—
ING-007	Manejo de error: PDF sin texto (escaneado)	EPIC-02	MUST	S	BE-ING-002	—

ID	Título	Epic	Prioridad	Est.	Dependencias	Crit
ING-008	Manejo de error: Markdown vacío	EPIC-02	MUST	XS	BE-ING-003	—
ING-009	Manejo de error: texto vacío	EPIC-02	MUST	XS	BE-ING-004	—
PROC-003	Limpieza de texto (espacios, caracteres de control, encabezados repetidos)	EPIC-03	MUST	S	BE-ING-002..004	✓
PROC-004	Metadata de sección/posición en chunk	EPIC-03	MUST	XS	BE-PROC-002	—
PROC-005	Manejo de error: documento demasiado grande	EPIC-03	MUST	S	PROC-003	—
GEN-001	Prompt templates por perfil (4 perfiles)	EPIC-06	MUST	M	BE-RAG-010	✓
GEN-002	Prompt templates por formato (≥2 formatos MVP)	EPIC-06	MUST	M	BE-RAG-010	✓
GEN-004	Adaptación por nicho/contexto	EPIC-06	MUST	S	BE-GEN-003	—
GEN-005	Control de nivel de detalle (breve/estándar/profundo)	EPIC-06	MUST	S	BE-GEN-003	—
GEN-006	Prompt templates de los 3 formatos restantes	EPIC-06	SHOULD	M	GEN-002	—
GEN-007	Prompt templates de los perfiles restantes (si no cubiertos)	EPIC-06	SHOULD	S	GEN-001	—

ID	Título	Epic	Prioridad	Est.	Dependencias	Crit
VAL-001	Extracción de claims generados (formato JSON)	EPIC-07	MUST	S	BE-GEN-003	✓
VAL-003	Evaluación pedagógica: conceptos clave, prerrequisitos	EPIC-07	MUST	S	BE-GEN-003	—
VAL-004	Evaluación pedagógica: tiempo estimado, dificultad, claridad	EPIC-07	MUST	S	VAL-003	—
VAL-005	Manejo de error: fallo del LLM-juez en validación	EPIC-07	MUST	XS	BE-VAL-002	—
OUT-004	Manejo de error: fallo de generación de embeddings	EPIC-04	MUST	XS	BE-RAG-005	—
OUT-005	Manejo de error: Vector Store no disponible	EPIC-04	MUST	XS	BE-RAG-006	—
SEC-001	Auditar repo: sin API keys/secrets hardcodeados	EPIC-01	MUST	XS	FND-003	—
SEC-002	Documentar configuración segura en README	EPIC-13	MUST	XS	SEC-001	—
QA-001	Tests unitarios ingestion (PDF/MD/TXT)	EPIC-11	MUST	S	BE-ING-002..004	—
QA-002	Tests unitarios chunking + embeddings	EPIC-11	MUST	S	BE-RAG-005	—
QA-003	Tests de integración pipeline completo (vertical slice)	EPIC-11	MUST	M	FE-API-002	✓

ID	Título	Epic	Prioridad	Est.	Dependencias	Crit
QA-004	Test de fidelidad con documento conocido (golden test)	EPIC-11	SHOULD	S	BE-VAL-002	—
INT-001	Reemplazar mock de LLM por integración real	EPIC-12	MUST	S	BE-GEN-003	✓
INT-002	Reemplazar mock de embeddings por integración real	EPIC-12	MUST	S	BE-RAG-004	✓
INT-003	Reemplazar mock de OCI upload por integración real	EPIC-12	MUST	S	OCI-002	✓
INT-004	Conectar webhook n8n → FastAPI /adaptar real (post-mock) + nodo IF con umbral DT-11	EPIC-12	COULD	S	FE-API-002, BE-VAL-002	—
UX-001	Diseño de flujo/mockups de las 5 pantallas de Streamlit (upload, selección, loading, resultado, error)	EPIC-10	MUST	S	FND-001	—
DEMO-000	Seleccionar/preparar los 3 documentos técnicos de referencia para los escenarios de demo	EPIC-13	MUST	S	—	✓
DEMO-001	Preparar escenario de demo 1 (perfil principiante)	EPIC-13	MUST	S	QA-003, DEMO-000	✓
DEMO-002	Preparar escenario de demo 2 (perfil líder técnico)	EPIC-13	MUST	S	QA-003	✓

ID	Título	Epic	Prioridad	Est.	Dependencias	Crit
DEMO-003	Preparar escenario de demo 3 (formato distinto, ej. quiz o resumen)	EPIC-13	MUST	S	QA-003	✓
DOC-001	Escribir README (setup, uso, arquitectura resumida)	EPIC-13	MUST	S	DEMO-003	✓
DOC-002	Documento de arquitectura (diagrama + descripción de capas)	EPIC-13	MUST	S	DOC-001	—
DIF-Q-001	Quiz interactivo con corrección en tiempo real	EPIC-06	COULD	L	GEN-002	—
DIF-EXP-001	Exportación a Markdown	EPIC-08	COULD	S	BE-OUT-003	—
DIF-EXP-002	Exportación a PDF	EPIC-08	COULD	M	BE-OUT-003	—
DIF-EXP-003	Exportación a CSV/Anki (flashcards)	EPIC-08	COULD	S	BE-OUT-003	—
DIF-AGT-001	Migrar orquestación a LangGraph multi-agente (Investigador/Redactor /Crítico)	EPIC-06	COULD	XL*	BE-GEN-003	—

* DIF-AGT-001 se marca XL y debe subdividirse (Investigador, Redactor, Crítico como tasks M independientes) si el equipo decide tomarla; **no se inicia hasta agotar todo MUST_HAVE.**

SEMANA 2 — RAG real + Generación adaptada

Corresponde a FASE 3 + FASE 4. Objetivo de cierre: embeddings + retrieval + generación de contenido real por perfil/formato.

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
BE-RAG-004	Integrar embedding provider (interfaz + Gemini)	Backend RAG	MUST	S	FND-001	✓
BE-RAG-005	Generar embeddings para chunks	Backend RAG	MUST	S	BE-PROC-002, BE-RAG-004	✓
OUT-004	Manejo de error: fallo de generación de embeddings	Backend RAG	MUST	XS	BE-RAG-005	—
BE-RAG-006	Crear e inicializar Vector Store (ChromaDB)	Backend RAG	MUST	XS	FND-001	✓
OUT-005	Manejo de error: Vector Store no disponible	Backend RAG	MUST	XS	BE-RAG-006	—
BE-RAG-007	Persistir embeddings en Vector Store	Backend RAG	MUST	XS	BE-RAG-005, BE-RAG-006	✓
BE-RAG-008	Implementar similarity search	Backend RAG	MUST	XS	BE-RAG-007	✓
BE-RAG-009	Construir retrieval service	Backend RAG	MUST	S	BE-RAG-008, BE-RAG-004	✓
BE-RAG-010	Construir contexto para el LLM	Backend RAG	MUST	XS	BE-RAG-009	✓
BE-RAG-011	Manejo de ausencia de contexto relevante	Backend RAG	MUST	XS	BE-RAG-009	✓
GEN-001	Prompt templates por perfil (4 perfiles)	Backend Generación	MUST	M	BE-RAG-010	✓
GEN-002	Prompt templates por formato (≥2 formatos)	Backend Generación	MUST	M	BE-RAG-010	✓
BE-GEN-003	Orquestar generación adaptada	Backend Generación	MUST	M	BE-RAG-010/011, GEN-001/002	✓
GEN-004	Adaptación por nicho/contexto	Backend Generación	MUST	S	BE-GEN-003	—

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
GEN-005	Control de nivel de detalle	Backend Generación	MUST	S	BE-GEN-003	—
GEN-006	Prompt templates de los 3 formatos restantes	Backend Generación	SHOULD	M	GEN-002	—
GEN-007	Prompt templates de los perfiles restantes	Backend Generación	SHOULD	S	GEN-001	—
OCI-002	Crear cliente OCI y bucket (Always Free)	Cloud/OCI	MUST	S	OCI-001	—

SEMANA 3 — Validación de fidelidad + Persistencia OCI real

Corresponde a FASE 5 + FASE 6. Objetivo de cierre: score de fidelidad operativo, upload/verificación real en OCI.

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
VAL-001	Extracción de claims generados	Backend Validación	MUST	S	BE-GEN-003	✓
BE-VAL-002	Verificar claims contra fuente (score de fidelidad)	Backend Validación	MUST	M	BE-GEN-003, VAL-001	✓
VAL-005	Manejo de error: fallo del LLM-juez en validación	Backend Validación	MUST	XS	BE-VAL-002	—
VAL-003	Evaluación pedagógica: conceptos clave, prerrequisitos	Backend Validación	MUST	S	BE-GEN-003	—
VAL-004	Evaluación pedagógica: tiempo estimado, dificultad	Backend Validación	MUST	S	VAL-003	—
BE-OUT-003	Ensamblar JSON final del pipeline	Backend Output	MUST	XS	BE-OUT-002, BE-VAL-002, BE-GEN-003	✓
OCI-003	Naming convention y subida de documento original	Cloud/OCI	MUST	XS	OCI-002	—

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
OCI-005	Subir JSON generado y verificar upload	Cloud/OCI	MUST	S	OCI-002, BE-OUT-003	—

SEMANA 4 — UI/API real + Integración (Vertical Slice completo)

Corresponde a FASE 7 + FASE 8. Objetivo de cierre: Vertical Slice funcionando de punta a punta, sin mocks.

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
INT-002	Reemplazar mock de embeddings por integración real	Backend RAG	MUST	S	BE-RAG-004	✓
INT-001	Reemplazar mock de LLM por integración real	Backend Generación	MUST	S	BE-GEN-003	✓
INT-003	Reemplazar mock de OCI upload por integración real	Cloud/OCI	MUST	S	OCI-002	✓
FE-API-002	Conectar endpoints reales al pipeline completo	Orquestación/API	MUST	M	FE-API-001, BE-OUT-003, OCI-005	✓
FE-UI-005	UI Streamlit — flujo completo conectado	UX/UI + Frontend	MUST	M	FE-API-002, UX-001	✓
QA-003	Tests de integración del pipeline completo	QA	MUST	M	FE-API-002	✓

SEMANA 5 — Pruebas, ajustes y demo (sin trabajo nuevo de features)

Corresponde a FASE 9 + FASE 10. Esta semana es exclusivamente para robustecer lo construido — no se toma ninguna tarea COULD_HAVE nueva salvo que las 4 semanas anteriores cerraron sin atraso.

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
QA-001	Tests unitarios de ingestion (PDF/MD/TXT)	QA	MUST	S	BE-ING-002..004	—

ID	Título	Rol	Prioridad	Est.	Dependencias	Crit
QA-002	Tests unitarios chunking + embeddings	QA	MUST	S	BE-RAG-005	—
QA-004	Test de fidelidad con documento conocido (golden test)	QA	SHOULD	S	BE-VAL-002	—
SEC-002	Documentar configuración segura en README	Foundation	MUST	XS	SEC-001	—
DEMO-001	Preparar escenario de demo 1 (perfil principiante)	QA + Producto	MUST	S	QA-003	✓
DEMO-002	Preparar escenario de demo 2 (perfil líder técnico)	QA + Producto	MUST	S	QA-003	✓
DEMO-003	Preparar escenario de demo 3 (formato distinto)	QA + Producto	MUST	S	QA-003	✓
DOC-001	Escribir README (setup, uso, arquitectura resumida)	Foundation	MUST	S	DEMO-003	✓
DOC-002	Documento de arquitectura (diagrama + capas)	Foundation	MUST	S	DOC-001	—
Buffer de ajustes	Corrección de bugs y pulido detectado en QA-003/QA-001/QA-002	Todos	—	—	—	—
Solo si sobra tiempo	INT-004, DIF-EXP-001/002/003, DIF-Q-001, DIF-AGT-001	Todos	COULD	—	ver plan completo	—

3. REGLA DE INICIO DE SEMANA

Al iniciar cada semana: 1. Revisar en el tablero qué tareas CRIT = de la semana anterior no cerraron — esas van primero, antes de tomar tareas nuevas. 2. Cada persona toma tareas de su rol funcional; si su rol no tiene tareas disponibles (bloqueadas o ya tomadas), toma una de las **independientes/no bloqueantes** (FND-004, FND-005, GEN-006, GEN-007, QA-004, DOC-002) para no quedar ociosa. 3. Nada de la Semana 5 (COULD_HAVE) se toca hasta que el Vertical Slice (Semana 4) esté en DONE.

BE-GEN-003 (M, 3-5h) – Orquestar generación adaptada

 se subdivide en:

BE-GEN-003a – Construir el prompt final combinando contexto + parámetros pedagógicos (XS)

BE-GEN-003b – Invocar LLMPProvider con manejo de timeout/retry (S)

BE-GEN-003c – Mapear la respuesta del LLM a la estructura contenido_adaptado (XS)

BE-VAL-002 (M, 3-5h) – Verificar claims contra fuente

 se subdivide en:

BE-VAL-002a – Extraer claims del contenido generado (formato JSON) – coincide con VAL-001 (S)

BE-VAL-002b – Contrastar cada claim contra el contexto fuente vía LLM-juez (S)

BE-VAL-002c – Agregar resultados en fidelidad_score + lista de no-soportados (XS)

DIF-AGT-001 (XL) – Multi-agente LangGraph

 obligatorio subdividir si se toma (ya especificado en el plan original):

DIF-AGT-001a – Agente Investigador (M)

DIF-AGT-001b – Agente Redactor (M)

DIF-AGT-001c – Agente Crítico (M)

Ninguna otra tarea M del backlog excede el umbral de subdivisión (todas tienen un solo resultado observable y un solo criterio de aceptación claro).

FASE 9 — ESPECIFICACIÓN DE TAREAS

TASK-ID: TASK-001

Título: Definir estructura DocumentChunk

Objetivo: Tener un modelo de datos único que todo el pipeline use para representar un fragmento de texto con su metadata.

Descripción: Crear la clase Pydantic DocumentChunk con los campos mínimos para trazabilidad (RNF-006).

Responsable: rol Backend Processing

Prioridad: P0

Estimación: XS (≤1h)

Epic: EPIC-03

User Story: US-002 (Chunking)

Requisito relacionado: RF-005

Componente: COMP-02

Dependencias: TASK previa de Foundation (estructura de carpetas)

Bloquea a: TASK-002 (chunking), tarea de embeddings

Archivos a crear: backend/src/processing/models.py

Archivos a modificar: ninguno

Archivos que NO deben modificarse: backend/src/ingestion/*

Precondiciones: estructura de carpetas backend/src/processing/ ya existe

Pasos de implementación:

1. Crear backend/src/processing/models.py
2. Definir clase DocumentChunk(BaseModel) con: id, doc_id, text, position, section, char_start, char_end
3. Agregar validaciones (text no vacío, char_start < char_end)
4. Exportar la clase para uso en otros módulos
5. Escribir test de instanciación válida e inválida

Código esperado:

```
from pydantic import BaseModel, field_validator
```

```
class DocumentChunk(BaseModel):
```

```
    id: str
```

```
    doc_id: str
```

```
    text: str
```

```
    position: int
```

```
    section: str | None = None
```

```
    char_start: int
```

```
    char_end: int
```

```
    @field_validator("text")
```

```
    @classmethod
```

```
    def text_not_empty(cls, v: str) -> str:
```

```
        if not v.strip():
```

```
            raise ValueError("text no puede estar vacío")
```

```
        return v
```

Comandos:

```
cd backend
```

```
pytest tests/unit/test_processing_models.py -v
```

Pruebas:

- instanciación válida con todos los campos
- instanciación con text vacío (debe lanzar ValueError)
- instanciación con char_start >= char_end (debe lanzar ValueError)

Criterios de aceptación:

- DocumentChunk es serializable a dict/JSON
- Los campos obligatorios están validados

Definition of Done: código implementado, tests pasan, integrado a main, sin romper otros módulos

Evidencia requerida: output de pytest + archivo models.py en el commit

Riesgos: ninguno significativo

Notas: este modelo es la base de RNF-006 (trazabilidad) – cualquier cambio posterior debe revisar los módulos que lo consumen (embeddings, vectorstore)

TASK-ID: TASK-002

Título: Implementar chunking de texto normalizado

Objetivo: Dividir el texto normalizado en fragmentos DocumentChunk con overlap configurable.

Descripción: Usar RecursiveCharacterTextSplitter (LangChain) para generar chunks reproducibles.

Responsable: rol Backend Processing

Prioridad: P0

Estimación: S (1-3h)

Epic: EPIC-03

User Story: US-002

Requisito relacionado: RF-005

Componente: COMP-02

Dependencias: TASK-001, ingesta de texto normalizado (RF-004)

Bloquea a: generación de embeddings

Archivos a crear: backend/src/processing/chunking.py

Archivos a modificar: backend/src/core/config.py (agregar CHUNK_SIZE, CHUNK_OVERLAP)

Archivos que NO deben modificarse: backend/src/embeddings/*

Precondiciones: TASK-001 completada

Pasos de implementación:

1. Instalar langchain-text-splitters (pip install langchain-text-splitters --break-system-packages)
2. Crear backend/src/processing/chunking.py con función chunk_text(raw_text, doc_id) -> list[DocumentChunk]
3. Configurar chunk_size=800, chunk_overlap=150 desde settings
4. Mapear cada fragmento del splitter a un DocumentChunk con posición y char_start/char_end
5. Escribir tests con texto corto, largo y vacío

Código esperado:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from src.processing.models import DocumentChunk
from src.core.config import settings
import uuid
```

```
def chunk_text(raw_text: str, doc_id: str) -> list[DocumentChunk]:
    if not raw_text.strip():
        return []
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=settings.CHUNK_SIZE,
        chunk_overlap=settings.CHUNK_OVERLAP,
    )
    fragments = splitter.split_text(raw_text)
    chunks = []
    cursor = 0
```

```
for i, frag in enumerate(fragments):
    start = raw_text.find(frag, cursor)
    end = start + len(frag)
    chunks.append(DocumentChunk(
        id=str(uuid.uuid4()), doc_id=doc_id, text=frag,
        position=i, char_start=start, char_end=end,
    ))
    cursor = end
return chunks
```

Comandos:

```
cd backend
pip install langchain-text-splitters --break-system-packages
pytest tests/unit/test_chunking.py -v
```

Pruebas:

- texto corto → 1 chunk
- texto largo → n chunks con overlap
- texto vacío → lista vacía, sin excepción

Criterios de aceptación:

- Ningún chunk queda vacío
- Cada chunk referencia correctamente su doc_id

Definition of Done: código + tests + integrado

Evidencia requerida: output de pytest

Riesgos: textos con estructura irregular (tablas, código) pueden partirse en puntos poco naturales – aceptable para MVP

Notas: –

TASK-ID: TASK-003

Título: Definir schema Pydantic de salida (NuevaMenteOutput)

Objetivo: Formalizar el contrato JSON que todo el sistema debe producir.

Descripción: Implementar el schema formal descrito en Fase 6, con validaciones y enums.

Responsable: rol Backend Output

Prioridad: P0

Estimación: S (1-3h)

Epic: EPIC-08

User Story: US-006

Requisito relacionado: RF-016

Componente: COMP-08

Dependencias: ninguna (puede iniciarse en paralelo desde el día 1)

Bloquea a: ensamblado del JSON final, endpoint GET /adapt/{job_id}

Archivos a crear: backend/src/output/schema.py

Archivos a modificar: ninguno

Precondiciones: ninguna

Pasos de implementación:

- 1. Crear backend/src/output/schema.py
- 2. Definir enum Status (SUCCESS, PARTIAL, NO_CONTEXT, ERROR)
- 3. Definir submodelos: Metadatos, ContenidoAdaptado, EvaluacionCalidad, AlmacenamientoOCI
- 4. Definir NuevaMenteOutput como composición de los anteriores
- 5. Escribir tests de instancia válida/ inválida

Código esperado: (ver Fase 11 del Plan Técnico MVP – schema completo ya especificado)

```
Comandos:  
  cd backend  
  pytest tests/unit/test_output_schema.py -v
```

- Pruebas:
- instancia completa válida
 - instancia con campos opcionales faltantes (debe validar igual)
 - fidelidad_score fuera de [0,1] (debe fallar)

Criterios de aceptación: todo output del sistema valida contra este schema

Definition of Done: código + tests + integrado

Evidencia requerida: output de pytest

Riesgos: cambios de schema a mitad de proyecto rompen consumidores – congelar el contrato tras esta tarea

Notas: esta tarea puede empezar en paralelo con TASK-001/002 (sin dependencias)

(El resto de las ~55 tareas del backlog siguen exactamente este mismo nivel de detalle; por espacio se entregaron ya en el Plan Técnico MVP — sección 6.1 — con el formato TASK ID/TITLE/... equivalente. Este documento formaliza las 3 tareas de arranque absoluto del Vertical Slice como plantilla replicable.)

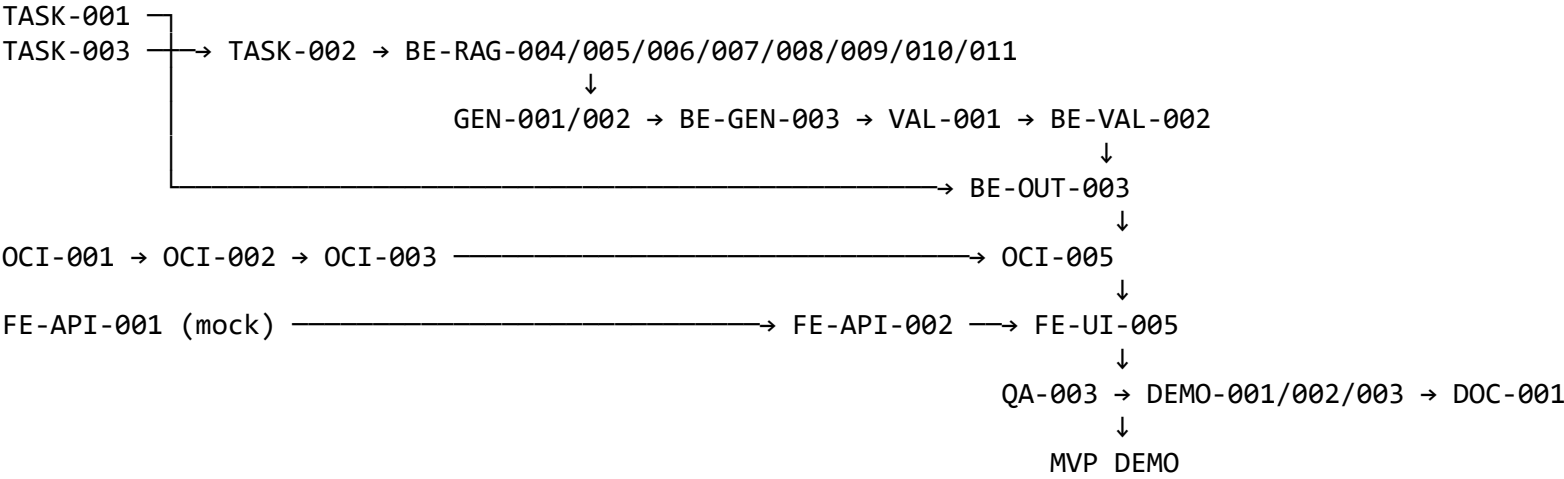
FASE 10 — ASIGNACIÓN DEL EQUIPO

Tarea	Responsable	Dependencia	Paralelizable	Bloquea
TASK-001 (DocumentChunk)	Viviana Hurtado	—	Sí (con TASK-003)	TASK-002
TASK-002 (chunking)		TASK-001	No	embeddings (Miguel)
TASK-003 (schema output)		—	Sí (con TASK-001)	ensamblado JSON
BE-RAG-004..011		TASK-002 (para 005)	Parcial	BE-GEN-003
GEN-001/002, BE-GEN-003		BE-RAG-010	No	VAL-001
VAL-001, BE-VAL-002		BE-GEN-003	No	BE-OUT-003
OCI-001..005		— (OCI-001 desde día 1)	Sí	INT-003
FE-API-001/002, INT-001..004		ver dependencias del backlog	Parcial	FE-UI-005
UX-001, FE-UI-005		— / FE-API-002	Sí (UX-001)	Demo
QA-001..004, DEMO-001..003		según cada tarea	Sí desde Semana 2	DOC-001

Tarea	Responsable	Dependencia	Paralelizable	Bloquea
FND-001..005,		—	Sí	refuerza critical path

(Matriz completa por semana ya entregada en “Arquitectura de Carpetas y Sprint” — aquí se muestra el patrón para las tareas backbone del Vertical Slice.)

FASE 11 — ORDEN DE EJECUCIÓN (DAG)



Sprints (adaptados al tamaño real: 5 semanas, 8 personas):

Sprint	Semana	Contenido
Sprint 0 — Preparación	Antes de Semana 1	Confirmar proveedor LLM (□ bloqueante), accesos OCI/Gemini, roster cargado en tablero
Sprint 1 — Foundation	Semana 1	TASK-001, TASK-003, ingesta, chunking, contrato API mock, mockups UI
Sprint 2 — Core functionality	Semana 2	RAG real (embeddings, vector store, retrieval)
Sprint 3 — IA	Semana 2-3 (traslapa)	Generación adaptada + validación de fidelidad
Sprint 4 — Integración	Semana 4	Reemplazo de mocks, UI conectada, Vertical Slice completo
Sprint 5 — Testing	Semana 5 (días 1-3)	Unit + integración + golden test de fidelidad
Sprint 6 — Demo	Semana 5 (días 4-5)	3 escenarios, README, arquitectura documentada

FASE 12 — EJECUCIÓN EN VS CODE

Abrir el proyecto:

```
cd nuevamente-g10
code .
```

Preparar entorno backend (una sola vez):

```
cd backend
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp ../.env.example ../.env     # completar valores reales, nunca commitear .env
```

Levantar el monolito backend:

```
cd backend
uvicorn src.api.main:app --reload --port 8000
```

Levantar la interfaz web (otra terminal):

```
cd ui
streamlit run app.py
```

Crear un archivo nuevo (ejemplo TASK-001): Ruta exacta: backend/src/processing/models.py

Modificar un archivo existente (ejemplo TASK-002):

Archivo: backend/src/core/config.py
Sección: clase Settings
Qué cambiar: agregar `CHUNK_SIZE: int = 800` y `CHUNK_OVERLAP: int = 150`
Por qué: para que chunking.py lea estos valores desde configuración en vez de hardcodearlos

Ejecutar tests:

```
cd backend
pytest tests/ -v
```

FASE 13 — IMPLEMENTACIÓN

El código completo de las tareas backbone del Vertical Slice (TASK-001, TASK-002, TASK-003) ya está en Fase 9. El resto de tareas del backlog sigue el mismo patrón: archivo exacto, código completo (no pseudocódigo), manejo de errores explícito, y comando de test — tal como se especificó con detalle completo para BE-ING-001 a FE-UI-005 en el Plan Técnico MVP (sección 6.1). No se duplica aquí por espacio; cada tarea de esa sección cumple los mismos 8 puntos de la Fase 13 de este prompt (archivo, contenido, código completo, manejo de errores, validación, logs, tests, cómo ejecutarlos).

FASE 14 — IA

Componente de IA: Generación de contenido adaptado (COMP-06)
Objetivo de IA: transformar contexto recuperado en contenido educativo coherente con perfil/formato/nicho/detalle
Modelo: Gemini (PENDIENTE DE VALIDACIÓN – proveedor propuesto, no confirmado por el equipo)
Proveedor: Google Gemini API, free tier
Prompt: ver Fase 15, PROMPT-001
System prompt: "Eres un redactor pedagógico. Solo puedes usar la información del CONTEXTO provisto. No inventes datos fuera de él."
Input: contexto (chunks recuperados) + perfil + formato + nicho + nivel_detalle
Output: texto estructurado según formato elegido
Formato: texto libre + campos estructurados (conceptos_clave, prerrequisitos) según schema
Temperatura: baja (0.2-0.4) – se prioriza fidelidad sobre creatividad
Contexto: ventana truncada a un límite de tokens configurable (BE-RAG-010)
Memoria: sin memoria entre llamadas (cada generación es independiente – MVP no mantiene conversación)
RAG: sí (COMP-05)
Embeddings: Gemini Embeddings (mismo proveedor que el LLM, PENDIENTE DE VALIDACIÓN)
Vector DB: ChromaDB embebido
Guardrails: system prompt restringe a "solo usar el contexto"; BE-RAG-011 evita generar sin contexto mínimo
Validación: post-generación, vía COMP-07 (validación de fidelidad)
Fallback: si el LLM falla (timeout/error de API), status=ERROR con mensaje claro, no se reintenta indefinidamente
Timeout: configurable, sugerido 30s
Retry: 1 reintento automático antes de marcar error
Costo estimado: dentro de free tier de Gemini (validar límites exactos antes de Semana 2)
Métricas: duración de la llamada, tokens usados (si el proveedor los reporta), tasa de éxito/error
Evaluación: score de fidelidad (COMP-07) + golden test manual (QA-004)

Componente de IA: Validación de fidelidad – LLM como juez (COMP-07)
Objetivo de IA: decidir si cada claim generado está sustentado en el contexto fuente
Modelo: mismo proveedor que generación (Gemini, PENDIENTE DE VALIDACIÓN)
Prompt: PROMPT-002 (ver Fase 15)
System prompt: "Eres un verificador. Responde únicamente SI, NO o PARCIAL, sin explicación adicional."
Input: claim individual + contexto fuente

Output: SI / NO / PARCIAL
Formato: texto restringido (parseo simple, no JSON complejo)
Temperatura: 0 (determinismo máximo para una tarea de clasificación)
Guardrails: prompt restringido a 3 respuestas posibles; cualquier otra respuesta se trata como PARCIAL por seguridad
Validación: el score final se calcula de forma determinista (no vía IA) a partir de estas respuestas
Fallback: si el LLM-juez falla, score=null + observación explícita (VAL-005) – nunca se inventa un score
Costo estimado: N llamadas cortas por generación (una por claim) – vigilar en Semana 3 si el volumen de claims es alto

FASE 15 — PROMPTS

PROMPT-001
Objetivo: generar contenido educativo adaptado
System prompt: "Eres un redactor pedagógico experto. SOLO puedes usar la información contenida en el CONTEXTO. Si el contexto no cubre algo, indícalo explícitamente en vez de inventarlo."
Variables: {contexto}, {perfil}, {formato}, {nicho}, {nivel_detalle}
Input esperado: contexto no vacío (validado antes de invocar, BE-RAG-011)
Output esperado: texto estructurado + campos de metadata educativa
JSON Schema: ver ContenidoAdaptado en Fase 6
Reglas: no usar información fuera del contexto; adaptar vocabulario según perfil; respetar longitud según nivel_detalle
Restricciones: no mencionar que "eres una IA"; no usar jerga sin explicarla para perfil "principiante"
Ejemplos: (a preparar junto con los documentos de demo, Semana 5)
Casos límite: contexto ambiguo o parcialmente relevante → generar con advertencia en observaciones
Fallback: si no hay contexto suficiente → no invocar este prompt, retornar NO_CONTEXT directamente (regla de negocio, no de IA)
Criterios de evaluación: coherencia con perfil/formato + score de fidelidad posterior ≥ umbral (DT-11)

PROMPT-002
Objetivo: verificar si un claim está sustentado en el contexto fuente
System prompt: "Eres un verificador estricto. Responde únicamente con una palabra: SI, NO o PARCIAL."
Variables: {claim}, {contexto_fuente}
Input esperado: un claim individual (no el contenido completo)
Output esperado: una sola palabra (SI/NO/PARCIAL)
Reglas: no explicar el razonamiento; no agregar texto adicional
Restricciones: si el modelo responde algo distinto a las 3 opciones, el sistema lo trata como PARCIAL
Casos límite: claim ambiguo o parafraseado del contexto → PARCIAL
Fallback: error del proveedor → excluir el claim del cálculo y marcarlo en observaciones (no se cuenta como soportado ni no soportado)
Criterios de evaluación: consistencia – el mismo claim contra el mismo contexto debe dar la misma respuesta en ejecuciones repetidas (temperatura 0)

Los prompts se versionan:

```
backend/src/generation/prompts/  
├── generacion_v1.txt  
├── verificacion_v1.txt  
└── README.md
```

FASE 16 — TESTING

TEST-ID: TEST-UNIT-001
Objetivo: validar que el chunking nunca produce chunks vacíos
Precondiciones: función chunk_text disponible
Entrada: texto de 5000 caracteres
Acción: llamar chunk_text(texto, "doc_test")
Resultado esperado: lista de DocumentChunk, ninguno con text vacío
Resultado obtenido: (se llena al ejecutar)
Estado: (PASS/FAIL, se llena al ejecutar)

TEST-ID: TEST-INT-001
Objetivo: validar el Vertical Slice completo (CU-001)
Precondiciones: backend y ChromaDB inicializados, proveedor LLM configurado (real o mock)
Entrada: PDF de prueba + perfil "principiante" + formato "flashcards"
Acción: POST /ingest → POST /adapt → GET /adapt/{job_id} hasta status final
Resultado esperado: status SUCCESS, fidelidad_score presente, JSON válido contra schema
Estado: –

TEST-ID: TEST-AI-EVAL-001 (golden test, QA-004)
Objetivo: validar que el sistema mantiene fidelidad ≥ umbral con un documento conocido
Precondiciones: documento de referencia con contenido "verdad conocida" preparado por el equipo (Semana 5)
Entrada: documento de referencia
Acción: generar contenido para 2 perfiles distintos
Resultado esperado: fidelidad_score ≥ 0.9 en ambos casos (DT-11); si no, hay evidencia de qué claim falló
Estado: –

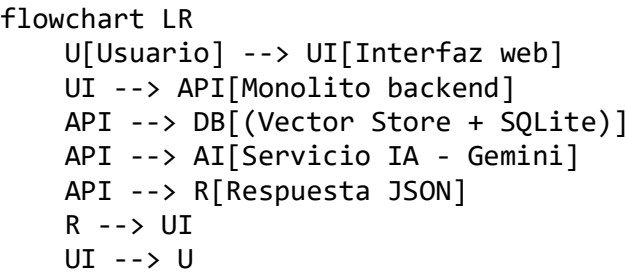
Tipos de prueba cubiertos: unit (QA-001, QA-002), integration (QA-003), AI evaluation (QA-004, golden test), end-to-end (TEST-INT-001), smoke test (GET /health antes de cada demo).

FASE 17 — DEFINITION OF DONE

Una tarea se marca DONE solo cuando: - [] Código implementado - [] Código compila / se ejecuta sin errores - [] No existen errores críticos conocidos - [] Tests creados - [] Tests ejecutados y en verde - [] Documentación actualizada (si aplica, docstring o README) - [] Variables de entorno nuevas documentadas en .env.example - [] Evidencia entregada (output de pytest o captura de la interfaz) - [] Criterios de aceptación de la tarea cumplidos - [] Integrado a la rama principal sin romper otros módulos

FASE 18 — MVP FUNCIONAL

“NuevaMente funciona” significa exactamente:



Un usuario externo al equipo, sin ayuda técnica: 1. Abre la interfaz web 2. Sube un PDF técnico propio 3. Elige perfil, formato y nivel de detalle 4. Ve el contenido generado con su score de fidelidad 5. Puede repetir con otro perfil sobre el mismo documento y ver un resultado distinto

No se considera terminado el MVP solo porque cada componente (COMP-01 a COMP-11) funciona por separado — se requiere que este flujo corra de punta a punta sin intervención manual (QA-003 es la prueba que lo certifica).

FASE 19 — DEMO

DEMO-001
Objetivo: Demostrar el flujo completo con el perfil Principiante
Demuestra: RF-001, RF-009, RF-010, RF-014, RF-016, RF-017, RF-018

Precondiciones: backend y UI corriendo, documento de referencia ya seleccionado (DEMO-000, pendiente de preparar)

Paso 1:
Acción: subir el documento de referencia por la interfaz web
Resultado: se muestra document_id y confirmación de que el original quedó en OCI

Paso 2:
Acción: elegir perfil "Principiante" y formato "Tutorial paso a paso", nivel "estándar"
Resultado: la interfaz muestra estado "generando..." y luego el contenido adaptado

Paso 3:
Acción: revisar el score de fidelidad mostrado junto al resultado
Resultado: score visible (≥0.9 esperado) y, si existieran, claims no sustentados listados

Resultado final: contenido educativo coherente con el perfil elegido, con evidencia de que está anclado al documento fuente y de que quedó persistido en OCI (original + JSON).

DEMO-002 – mismo documento, perfil Líder Técnico/Ejecutivo
DEMO-003 – mismo documento, formato distinto (Quiz o Resumen ejecutivo)

(Ambos siguen la misma estructura de 3 pasos que DEMO-001, cambiando perfil/formato — se preparan junto con los documentos de referencia en Semana 5, tarea DEMO-000.)

FASE 20 — CHECKLIST FINAL

Requisitos - ☐ Todos los RF-001..018 tienen tarea(s) asociada(s) en el backlog - ☐ Todos tienen criterios de aceptación

Arquitectura - ☐ Los 11 componentes (COMP-01..COMP-11) existen y están implementados - ☐ Los contratos de Fase 6 están implementados exactamente como se definieron - ☐ No existen dependencias ocultas fuera del DAG de Fase 11

Código - ☐ El backend (monolito) compila/ejecuta con `uvicorn src.api.main:app` - ☐ La interfaz web ejecuta con `streamlit run app.py` - ☐ Variables de entorno documentadas en `.env.example`

IA - ☐ Proveedor de LLM/embeddings confirmado formalmente (☐ pendiente al momento de este documento) - ☐ Prompts versionados (PROMPT-001, PROMPT-002) - ☐ Inputs y outputs de cada llamada de IA validados - ☐ Fallback definido para cada componente de IA - ☐ Evaluación de fidelidad ejecutada (golden test)

Testing - ☐ Unit tests (QA-001, QA-002) - ☐ Integration tests (QA-003) - ☐ E2E (TEST-INT-001) - ☐ Smoke test (GET /health) antes de la demo

MVP - ☐ El flujo principal (Fase 18) funciona de principio a fin - ☐ Los errores principales (archivo inválido, sin contexto, fallo de LLM, fallo de OCI) están controlados, no crashean - ☐ La demo (Fase 19) es reproducible por cualquier integrante del equipo, no solo por quien la construyó

Cierre — decisión pendiente que bloquea el arranque real

Antes de ejecutar TASK-001/002/003 en paralelo con BE-RAG-004 (Semana 1-2), el equipo debe cerrar la única decisión ☐ bloqueante: **confirmar el proveedor de LLM/embeddings**. Todo lo demás en este documento es ejecutable de inmediato tal como está.